

Final Project

INTRODUCTION

The aim of this project is to build a machine learning model that predicts the probability that a home team is going to win an NFL (National Football League) game given a current game state (features). A game state is described by statistics of a team's field position (yards to go, yards to endzone, yards gained), game's time remaining (quarter, time_remaining), and the score difference(home score, away score). Another key feature is the offensive and defensive team strengths based on each team's historical performance (offense strength and defense strength). All of these variables capture both the immediate context of the game and the relative strengths of the teams, and are leveraged by the model to predict the win probability (model's output). A binary target outcome, whether the home team wins the game, contributes to the model training as well. The model is trained on past data to predict the win probability at any point of time in the game, which can be valuable for fans, analysts, coaches or strategists seeking to better understand the game dynamics, evaluate team performance, and support real-time decision making in sports analytics.

METHODS

Data Collection

The dataset was extracted using the ESPN Public API (<https://github.com/pseudo-r/Public-ESPN-API>). To start, the API was used to collect all NFL games from the 2022 and 2023 season, providing two datasets containing 336 games and 349 games respectively with 5 variables:

Variable	Description	Type
id	Game ID number	String
name	Game's name (teams playing)	String
home_team	Home team's ID number	String
away_team	Away team's ID number	String

Variable	Description	Type
date	Game's date	String

Table 1: Games Dataset

Next, a different endpoint of the ESPN Public API was used to retrieve play-by-play data that occurred in each game of the 2022 and 2023 season. This produced two dataset of 57,547 observations (or single plays) and 52,703 observations respectively with 13 variables:

Variable	Description	Type
gid	Game ID number	Integer
pid	Play ID number	Integer
play_action	Description of the play	String
quarter	Quarter in which the play occurred	Integer
time_remaining	Time remaining in the current quarter (seconds)	Float
down	Down in which the play occurred	Integer
yards_to_go	Yards needed to get a first down	Integer
yardsToEndzone	Yards needed to reach the endzone	Integer
yards_gained	Yards gained on the play	Integer
home_score	Home team's current score	Integer
away_score	Away team's current score	Integer
offense_team	Offensive team ID	Float
defense_team	Defensive team ID	Float

Table 2: Plays Dataset

df_plays_2023.to_csv("nfl_plays_2023.csv", index = False)

```
```{python}
#import the plays dataset from the data folder
train_data = pd.read_csv("data/nfl_plays_2022.csv")
test_data = pd.read_csv("data/nfl_plays_2023.csv")
```

## Data Cleaning & Wrangling

First, notice that games dataset's column "id" corresponds to the plays dataset's column "gid". However, they have different variable types, so "gid" was converted to a string, matching "id".

Since only the plays dataset was downloaded and may not align with potential API updates for the games dataset, we filter out any games that are not in the plays dataset using the “gid” set.

```
df_games_2022["id"] = df_games_2022["id"].astype(str)
df_games_2023["id"] = df_games_2023["id"].astype(str)

train_data["gid"] = train_data["gid"].astype(str)
test_data["gid"] = test_data["gid"].astype(str)

df_games_2022 = df_games_2022[df_games_2022["id"].isin(set(train_data["gid"]))]
df_games_2023 = df_games_2023[df_games_2023["id"].isin(set(test_data["gid"]))]
```

Next, based on the summary table of missing data, we observe that most variables in the plays dataset don’t have any missing values. The only variables with missing values are offense\_team and defense\_team.

Further investigation of these observations shows that these values are missing when the play occurred at the start or end of a quarter or game (indicated by variable play\_action). Since these plays don’t represent standard plays, they are not relevant for our model and thus are filtered out.

Moreover, after examining the summary statistics table, we see that all variables are within the reasonable ranges:

- home\_score and away\_score are positive (minimums are 0)
- yards\_gained has a range within -109 and 109 yards, which is the maximum length of a football field including the end zones
- time\_remaining between 0 and 900 which is 4 quarters of 15 minutes
- yards\_to\_go and yardsToEndzone are positive and under 100 yards, reflecting the maximum possible distance of a football field

However, the summary statistics table also show a minimum values of 0 for variables: down and quarter. After examining these observations, we see that these plays corresponds to kickoffs or start of a game. Because these events don’t represent standard play, we keep entries with quarter 1,2,3,4 and 5 (representing the 4 quarters of a standard NFL game and overtime), as well as downs 1,2,3,4 (representing offensive downs)

After filtering out irrelevant data, our datasets contains 52,410 and 48,376 meaningful plays for the training (2022) and testing (2023) datasets respectively.

```
#Train data
#train_data.describe() #check for extreme data
missing = train_data.isna().sum().to_frame("n_missing")
missing["pct_missing"] = (missing['n_missing']/len(train_data)).round(4)
#missing.sort_values("pct_missing", ascending=False).head(20) #check for missing data
```

```
#Train data
train_data_a = train_data[train_data['down']==0]
#display(train_data_a)

#shows that when downs = 0 => kickoff/start of game

train_data_b = train_data[train_data['quarter']==0]
#display(train_data_b)

#shows that when quarter = 0 => start of game

train_data_c = train_data[train_data['offense_team'].isna()]
#display(train_data_c)

#shows that when offense_team = 0 => start/end of quarter or game

train_data_d = train_data[train_data['defense_team'].isna()]
#display(train_data_d)

#shows that when defense_team = 0 => start/end of quarter or game
```

```
#Train data
check if quarter is either 1, 2, 3, 4
train_data = train_data[train_data['quarter'].isin([1,2,3,4,5])]
check if down is 1,2,3,4
train_data = train_data[train_data['down'].isin([1,2,3,4])]
offense and defense team is not null
train_data = train_data.dropna(subset=["offense_team", "defense_team"])

#train_data.info()
```

```
#Test data
#test_data.describe() #check for extreme data
missing = test_data.isna().sum().to_frame("n_missing")
missing["pct_missing"] = (missing['n_missing']/len(test_data)).round(4)
#missing.sort_values("pct_missing", ascending=False).head(20) #check for missing data
```

```

#Test data
test_data_a = test_data[test_data['down']==0]
#display(test_data_a)

#shows that when downs = 0 => kickoff/start of game

test_data_b = test_data[test_data['quarter']==0]
#display(test_data_b)

#shows that when quarter = 0 => start of game

test_data_c = test_data[test_data['offense_team'].isna()]
#display(test_data_c)

#shows that when offense_team = 0 => start/end of quarter or game

test_data_d = test_data[test_data['defense_team'].isna()]
#display(test_data_d)

#shows that when defense_team = 0 => start/end of quarter or game

```

```

#Test data
check if quarter is either 1, 2, 3, 4
test_data = test_data[test_data['quarter'].isin([1,2,3,4,5])]
check if down is 1,2,3,4
test_data = test_data[test_data['down'].isin([1,2,3,4])]
offense and defense team is not null
test_data = test_data.dropna(subset=["offense_team", "defense_team"])

#test_data.info()

```

## Feature Engineering

The target outcome for this model is a binary indicator of whether the home team wins the game. Since the dataset is rows of all plays played in the all games, we first grouped the plays per game, and find the maximum score for both the home and away team for each game. If the home team has the higher maximum score, then the target outcome variable, “home\_wins” is set to 1 indicating that the home team won. This target outcome will then be merged into the original plays dataset so that each row (or play) is assigned the final game outcome.

In order to better represent the state of the game, 4 additional variables were engineered from existing data:

- `gametime_remaining`: Time remaining in the whole game (instead of time remaining in the quarter as represented as the variable `time_remaning`) (in seconds)
  - **`gametime_remaining`** =  $((4 - \text{quarters}) \times 900) + \text{time\_remaining}$
  - where each quarter consists of 900 seconds ( = 15 minutes) and for quarter = 5 (indicating overtime plays), this value is set to NULL
- `score_diff`: Score Difference between home and away team (relative to the home team)
  - **`score_diff`** = `home_score` - `away_score`
  - the difference between the home team’s score and away team’s score at that time of play
- `offense_strength`: the strength of the team currently on offense
  - This value was calculated by the sum of wins / sum of games played per team
- `defense_strength`: the strength of the team currently on defense
  - This value was calculated by the sum of wins / sum of games played per team

Together, these engineered variables provide more context of the game, improving the model’s training to estimate the win probability.

Variable	Description	Type
<code>gid</code>	Game ID	String
<code>pid</code>	Play ID	Integer
<code>play_action</code>	Description of play	String
<code>quarter</code>	Quarter of play	Integer
<code>time_remaining</code>	Time remaining in quarter (seconds)	Float
<code>down</code>	Current down	Integer
<code>yards_to_go</code>	Yards to first down	Integer
<code>yardsToEndzone</code>	Distance to endzone	Integer
<code>yards_gained</code>	Yards gained on play	Integer
<code>home_score</code>	Home team score	Integer
<code>away_score</code>	Away team score	Integer
<code>offense_team</code>	Offensive team ID	String
<code>defense_team</code>	Defensive team ID	String
<code>gametime_remaining</code>	Time remaining in game (seconds)	Float
<code>score_diff</code>	Home score – away score	Integer
<code>home_wins</code>	Target variable (1 = home win, 0 = loss)	Integer
<code>offense_strength</code>	Offensive team win rate	Float
<code>defense_strength</code>	Defensive team win rate	Float

**Table 3: Final Dataset**

```
#Train data
#Feature Engineering
#Adding variable of time remaning in game instead of in quarter
train_data["gametime_remaining"] = np.where(train_data['quarter']<=4, (4-train_data['quarter']-1)*12, 0)

#Adding Score Difference relative to home team
train_data["score_diff"] = (train_data["home_score"] - train_data['away_score'])

#train_data.info()
#train_data.describe()
```

```
#Train data
train_data["offense_team"] = train_data["offense_team"].astype(int).astype(str)
train_data["defense_team"] = train_data["defense_team"].astype(int).astype(str)

#group train data by game id and take the maximum home team's score and away's team score
win_or_lose_2022 = train_data.groupby("gid").agg({"home_score": "max", "away_score": "max"})
1 indicates home team scores higher and 0 otherwise
win_or_lose_2022["home_wins"] = (win_or_lose_2022["home_score"] > win_or_lose_2022["away_score"])

win_or_lose_2022 = win_or_lose_2022.merge(df_games_2022[["id", "home_team", "away_team"]], left_on="gid", right_on="id", how="left")

train_data = train_data.merge(win_or_lose_2022[["gid", "home_wins"]], left_on="gid", right_on="gid", how="left")

games_outcome_2022 = []
for _, row in win_or_lose_2022.iterrows():
 if row["home_wins"] == 1:
 games_outcome_2022.append((row["home_team"], 1))
 games_outcome_2022.append((row["away_team"], 0))
 else:
 games_outcome_2022.append((row["home_team"], 0))
 games_outcome_2022.append((row["away_team"], 1))

games_outcome_2022_df = pd.DataFrame(games_outcome_2022, columns=['team', 'win'])

team_strength = games_outcome_2022_df.groupby("team")["win"].mean().reset_index()

train_data = train_data.merge(team_strength, left_on="offense_team", right_on="team", how="left")
train_data = train_data.merge(team_strength, left_on="defense_team", right_on="team", how="left")
```

```

#Test data
#Feature Engineering
#Adding variable of time remaning in game instead of in quarter
test_data["gametime_remaining"] = np.where(test_data['quarter']<=4, (4-test_data['quarter']))

#Adding Score Difference relative to home team
test_data["score_diff"] = (test_data["home_score"] - test_data['away_score'])

#test_data.info()
#test_data.describe()

```

```

#Test data
test_data["offense_team"] = test_data["offense_team"].astype(int).astype(str)
test_data["defense_team"] = test_data["defense_team"].astype(int).astype(str)

#group train data by game id and take the maximum home team's score and away's team score
win_or_lose_2023 = test_data.groupby("gid").agg({"home_score": "max", "away_score": "max"}).reset_index()
1 indicates home team scores higher and 0 otherwise
win_or_lose_2023["home_wins"] = (win_or_lose_2023["home_score"] > win_or_lose_2023["away_score"])

win_or_lose_2023 = win_or_lose_2023.merge(df_games_2023[["id", "home_team", "away_team"]], left_on="gid", right_on="id")

test_data = test_data.merge(win_or_lose_2023[["gid", "home_wins"]], left_on="gid", right_on="gid")

test_data = test_data.merge(team_strength, left_on="offense_team", right_on="team", how = "left")
test_data = test_data.merge(team_strength, left_on="defense_team", right_on="team", how = "left")

train_data.to_csv("train_data.csv", index = False)
test_data.to_csv("test_data.csv", index = False)

```

## EDA

### Statistics Summary of Key Variables

The summary table below shows key game statistics of 52,410 plays from the 2022 NFL season (train dataset) with each observation representing a single play including variables describing the game situation. The table confirms that all variables fall within expected ranges of a realistic game state.



	quarter	time_re- main- ing	down	yards	yard- sToEnd- yards	yards_gained	yards_lost	yards_net	game- time_re- main- ing	score	score_diff	of- ense	de- fense	strength
mean	2.59	407.24	2.02	8.41	51.36	5.77	11.72	10.17	1696.95	1.55	0.559	0.501	0.499	
std	1.13	275.30	1.01	4.12	24.42	10.43	9.67	9.16	1050.37	9.633	0.497	0.1449	0.1451	
min	1	0	1	1	0	-45	0	0	0	-37	0	0	0	
max	5	900	4	89	99	99	54	51	3600	47	1	1	1	

**Table 4: Summary Table of Key Variables**

```
#| echo: false
Counts of Plays per quarter
(ggplot(train_data, aes(x='factor(quarter)', fill='factor(down)')) +
geom_bar()+
labs(
 x='Quarter',
 y = "Number of Plays",
 fill = "Down",
 title = "Proportion of Downs per Quarter"
) +
theme_minimal()
)
```

**Figure 1: Proportion of Downs per Quarter**

This stacked bar graph displays the number of plays as well as the proportion of downs across each quarter. We can see that there are more plays occurring in second and fourth quarter which might be because teams tend to hustle as they approach a turnover or end of the game. Besides that, first and second down occurs more frequent compared to third and fourth down which is expected as they represent the start of offensive drives. Notice how there is more fourth down played in the fourth quarter compared to other quarters suggesting increased risk-taking drives attempted by teams to get more points in the limited time left. Another thing to highlight, is the low count in plays for 5th quarter as overtime don't happen often in NFL games. Hence, these findings suggests that both downs and quarters might influence win probability and thus are added as features of the model.

```
#| echo: false
Distribution of Time Remaining in each quarter
(ggplot(train_data, aes(x='factor(quarter)', y='time_remaining')) +
geom_boxplot()+
)
```

```
labs(
 x = "Time Remaining in Quarter (in seconds)",
 y = "Quarters",
 title = "Distribution of Time Remaining per Quarter"
) +
theme_minimal()
)
```

### Figure 2: Distribution of Time Remaining per Quarter

The distribution of time remaining within each quarter is balanced with plays occurring throughout the duration of the game. This suggests that plays are not concentrated in a certain time of the quarter and that our data is not skewed (it captures game states at all point in time of the game) . This is key for the model as it ensures that predictions can be made across all quarters of the game.

```
#!/ echo: false
#Distribution of Yards To Endzone
plt.hist(train_data["yardsToEndzone"], bins=100)
plt.xlabel("Yards to Enzone")
plt.ylabel("Count")
plt.title("Distribution of Yards To Endzone")
plt.show()
```

### Figure 3: Distribution of Yards to Endzone

Field position values are distributed across the field with more plays occurring at approximately 75 yards away from the end zone where most drives often begin. The wide spread indicates that the dataset captures a diverse range of game situation at every yard of the field, ensuring that predictions can be made by the model across the field.

```
#!/ echo: false
Association between GameTime Remaining and Score Difference
(ggplot(train_data, aes(x='gametime_remaining', y='score_diff')) +
geom_point(alpha=0.5)+
labs(
 x='Gametime Remaining (in Seconds)',
 y = "Score Difference (relative to Home team)",
 title = "Association between GameTime Remaining and Score Difference"
) +
theme_minimal()
)
```

**Figure 4: Association between Gametime Remaining and Score Difference** The association between the score difference and game time remaining seems to have a correlation where as game time decreases (game progresses), the score differences tend to increase in magnitude, reflecting the fact that games often become more decisive towards the end.

## Model Building

To search for the best model to predict the probability that the home team wins, both Random Forest classifier and the XGBoost Classifier were candidates, because: - Random Forest Classifier: this model creates decision trees by randomly creating subsets of the 8 key variables and takes the majority vote across the trees.

- XGBoost Classifier: this model creates a weak learner and slightly improves sequentially up

Both approaches improves prediction accuracy for a binary target outcome, whether the team wins (indicated by `home_wins = 1`) or loses (indicated by `home_wins = 0`) and outputs the probability of a home team win.

## Training and Testing

We have two datasets: - 2022 NFL games dataset which is our training dataset - 2023 NFL games dataset which is our testing dataset

The model is trained on the training set (2022 NFL games) and will be evaluated on the testing set (2023 NFL games) to evaluate its accuracy.

## Hyperparameter Tuning

To improve model performance, hyperparameters were tuned on the training set using 5-fold CV with random search. The following hyperparameters samples were considered: - Random Forest: - `max_features`: features per split ('sqrt', 1/3, 0.5) - `min_samples_leaf`: minimum samples or observation in each leaf (1, 5, 10)

- `n_estimators`: number of trees (set to 200)

- XGBoost:
  - `max_depth`: depth of each tree (4,6,8)
  - `learning_rate`: shrinkage per step (0.05, 0.1, 0.5)
  - `reg_lambda`: L2 regularization (0.1, 1, 10)
  - `reg_alpha`: L1 regularization (0, 0.1, 1)

- n\_estimators: number of trees (set to 200)

Here are the best hyperparameter combinations found for each model: - Random Forest: -  
max\_features: 'sqrt' - min\_samples\_leaf: 10 - n\_estimators: 200

- XGBoost:

- max\_depth: 6
- learning\_rate: 0.05
- reg\_lambda: 1
- reg\_alpha: 0.1
- n\_estimators: 200

```
features = ['gametime_remaining',"quarter", 'down', 'yards_to_go', "yardsToEndzone", "offense"]

X_train = train_data[features]
y_train = train_data['home_wins']

X_test = test_data[features]
y_test = test_data["home_wins"]
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import RandomizedSearchCV

param_grid = {
 "max_features": ['sqrt', 1/3, 0.5],
 "min_samples_leaf" : [1, 5, 10]
}

rf = RandomForestClassifier(
 n_estimators=200, random_state=65, n_jobs=-1
)
cv_rf = RandomizedSearchCV(
 rf, param_grid, cv=5, n_iter=10, n_jobs=-1,
 scoring="roc_auc"
)

cv_rf.fit(X_train, y_train)
print(f"Best params: "
 f"{cv_rf.best_params_}")
```

```

from xgboost import XGBClassifier

param_dist = {
 'max_depth': [4,6,8],
 'learning_rate': [0.05, 0.1, 0.5],
 'reg_lambda': [0.1, 1, 10],
 'reg_alpha': [0, 0.1, 1],
}

xgb = XGBClassifier(
 n_estimators=200,
 random_state=65,
 n_jobs=-1
)

xgb_search = RandomizedSearchCV(
 estimator=xgb,
 param_distributions=param_dist,
 n_iter=10 ,
 cv=5,
 scoring= 'roc_auc',
 random_state=65,
 n_jobs=-1,
 verbose=1
)

xgb_search.fit(X_train, y_train)

best_params_xgb = xgb_search.best_params_
print('Best params:', best_params_xgb)

```

## Model Evaluation

After the models were rebuild with the best combinations of hyperparameter above and fit on the training data, the models were ran on the test data and their performance are evaluated using:

- Accuracy: represents the model's ability to predict wins correctly
- F1-Score: represents the model's ability to balance precision and recall
- ROC-AUC: represents the model's performance in distinguishing a win or loss across all thresholds

Model	Test Accuracy	Misclassification Error	F1 Score	AUC
Random Forest	0.698	0.302	0.742	0.771
XGBoost	0.693	0.307	0.739	0.758

**Table 5: Model Performance Comparison**

## RESULTS

```

from sklearn.metrics import accuracy_score, f1_score, roc_auc_score, confusion_matrix, ConfusionMatrixDisplay
best_params_rf = cv_rf.best_params_

best_rf = RandomForestClassifier(
 **best_params_rf, n_estimators = 200, random_state=65, n_jobs=-1
)

best_rf.fit(X_train, y_train)

y_pred_rf = best_rf.predict(X_test)
y_prob_rf = best_rf.predict_proba(X_test)[: ,1]

rf_test_acc = accuracy_score(y_test, y_pred_rf)
rf_test_f1 = f1_score(y_test, y_pred_rf)
rf_test_auc = roc_auc_score(y_test, y_prob_rf)

print(f"Random Forest test accuracy: {rf_test_acc}")
print(f"Random Forest test F1-score: {rf_test_f1}")
print(f"Random Forest test AUC: {rf_test_auc}")

best_xgb = XGBClassifier(
 **best_params_xgb, n_estimators = 200, random_state=65, n_jobs=-1
)

best_xgb.fit(X_train, y_train)

y_pred_xgb = best_xgb.predict(X_test)
y_prob_xgb = best_xgb.predict_proba(X_test)[: ,1]

xgb_test_acc = accuracy_score(y_test, y_pred_xgb)

```

```

xgb_test_f1 = f1_score(y_test, y_pred_xgb)
xgb_test_auc = roc_auc_score(y_test, y_prob_xgb)

print(f"XGBoost test accuracy: {xgb_test_acc}")
print(f"XGBoost test F1-score: {xgb_test_f1}")
print(f"XGBoost test AUC: {xgb_test_auc}")

cm = confusion_matrix(y_test, y_pred_rf)
cm_xgb = confusion_matrix(y_test, y_pred_xgb)
fig, ax = plt.subplots(1, 2, figsize=(10, 8))

disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["Loss", "Win"])
disp.plot(ax=ax[0], cmap='Blues', xticks_rotation=45)
ax[0].set_title("Random Forest Classification")

disp_xgb = ConfusionMatrixDisplay(confusion_matrix=cm_xgb, display_labels=["Loss", "Win"])
disp_xgb.plot(ax=ax[1], cmap='Blues', xticks_rotation=45)
ax[1].set_title("XGBoost Classification")

plt.tight_layout()
plt.show()

```

**Figure 5: Confusion Matrices of Models** The Confusion Matrices shows diagonals shows the accuracy of both models. With greater proportion of the prediction (indicated by both the larger numbers and the darker shade of blue) lies at the true values (Predicted Win & True Win, Predicted Loss & True Loss). Notice that False Positives and False Negatives counts are significantly smaller than the True values, suggesting the high accuracy in the both models.

```

fig, ax = plt.subplots(1, 2, figsize=(10, 8))
rf_importances = best_rf.feature_importances_

imp_df_rf = pd.DataFrame({
 'feature': features,
 'importance': rf_importances
}).sort_values('importance')

xgb_importances = best_xgb.feature_importances_

imp_df_xgb = pd.DataFrame({
 'feature': features,
 'importance': xgb_importances
})

```

```

}).sort_values('importance')

ax[0].barh(imp_df_rf['feature'], imp_df_rf["importance"])
ax[0].set_title("Random Forest")

ax[1].barh(imp_df_xgb['feature'], imp_df_xgb["importance"])
ax[1].set_title("XGBoost")

plt.tight_layout()
plt.show()

```

**Figure 6: Feature Importance Graphs of Models** The Feature Importance Graphs highlights the top most influential features for each model. We see both models have score difference as the more dominant variable, followed by team strengths then remaining game time. This shows that not only does game statistics influence the model, but also each team’s relative strengths contributes meaningfully.

```

comparison_c = pd.DataFrame({
 'Model': ['Random Forest', "XGBoost"],
 'Test Accuracy': [rf_test_acc, xgb_test_acc],
 'Test Misclassification Error': [1-rf_test_acc, 1-xgb_test_acc],
 'Test F1-Score': [rf_test_f1, xgb_test_f1],
 'Test AUC': [rf_test_auc, xgb_test_auc]
})

print(comparison_c.to_string(index=False))

```

The final selected model is the Random Forest Classifier as it has a higher accuracy (0.698 vs 0.693) along with a higher F1-score (0.742 vs 0.739), indicating better balance between percision and recall, as well as higher AUC score (0.77 vs 0.76), making it more reliable in predicting both win and loss resutls.

With Random Forest Classifier as our model, we set a specific game, the “NFL 2024 game of the San Francisco 49ers vs the Washington Commanders” to provide an example of how our model could provide valuable insights.

Figure A, the plot shows how win probability of the home team (in this case, the Commanders) evovles throughout the game. At the start of the game (right hand side of the graph), the win probability is around 0.6 which is reasonable as we know that team strength affects the prediction quite significantly. As the game progresses, game state (e.g. score difference, field positions) changes and those changes influences the model’s prediction which is reflected in the fluctuation of the graph. Towards the end of the game (closer to gametime\_remaning = 0) the



win probability shifts to a more extreme value (0 or 1, in this case 0). This reflects a game state with a significant score difference of -17, limited time remaining and field position far away from the end zone (difficult to score a touchdown or a field goal), indicating there isn't much chances for the Commanders to possibly win at that game state.

Figure B highlights the difference in distribution of predicted win probabilities among the different quarters. In the early quarters, win probabilities centers around 0.6, reflecting high uncertainty due to ample remaining time. As the game progresses, the distribution spreads out in third quarter (a more uniform distribution). Another apparent insight is how the win probability in the fourth quarter concentrates in the two extreme values of 0 and 1, indicating increased certainty in game outcome as it gets closer to the end of the game and score differences becomes more influential.

Figure C illustrates the relationship between win probability and gametime remaining, highlighting the role of team's strength difference. At the early stages of the game, there are more extreme colour (purple or yellow) points towards the extreme values of 0 or 1, suggesting how the magnitude in strength difference influences the prediction of win probability. As the game progresses, the different colour points evens out, indicating that team strength difference doesn't influence the win probability as much as early in the game.

## CONCLUSION & SUMMARY

This project has developed a Random Forest Classifier Model that predicts whether a home team will win an NFL game given the game state, and outputs the win probability at that point of the game. The most influential features leveraged by the model are score difference, team strengths and game time remaining, as well as other less dominant features like field position also contribute in predicting the win probability.

The results shows that the model is able to produce meaningful and interpretable insights. For example, given a game state with significant negative score difference, limited time remaining and poor field position, the chances of winning are low and thus teams can take more risky plays to attempt to even out the score.

During model building, we also learn that a Random Forest Classifier performs slightly better than the XGBoost (by comparing the accuracy, F1-score and ROC-AUC). Hence, Random Forest Classifier was chosen as our final model with an accuracy of approximately 70%.

However, there are several limitations to this method. First, the model doesn't consider player statistics or whether they are playing or not, which can affect the offense or defense strength values, thus impacting game outcome. Besides that, the model doesn't consider team performance against specific opponents, and there is no distinction between the offense and defense strengths beyond the overall strength value.

Further improvements would include adding more features such as players conditions, importing more hisotrical data to capture team performance against specific teams and differentiating offense and defense strengths.

To summarize, this project shows that machine learning models can effectively estimate win probability in NFL games using conditions of the game. These findings can be valuable for understanding game dynamics and supporting decision-making in sports analytics, while also providing a foundation for future improvements.